



L LOVELY
P PROFESSIONAL
U UNIVERSITY

Project Report

Big Data Analytics Laboratory

Partitioning in Apache Hive

Submitted By:

Name: **Abhishek Sharma**

Registration No.: **12515331**

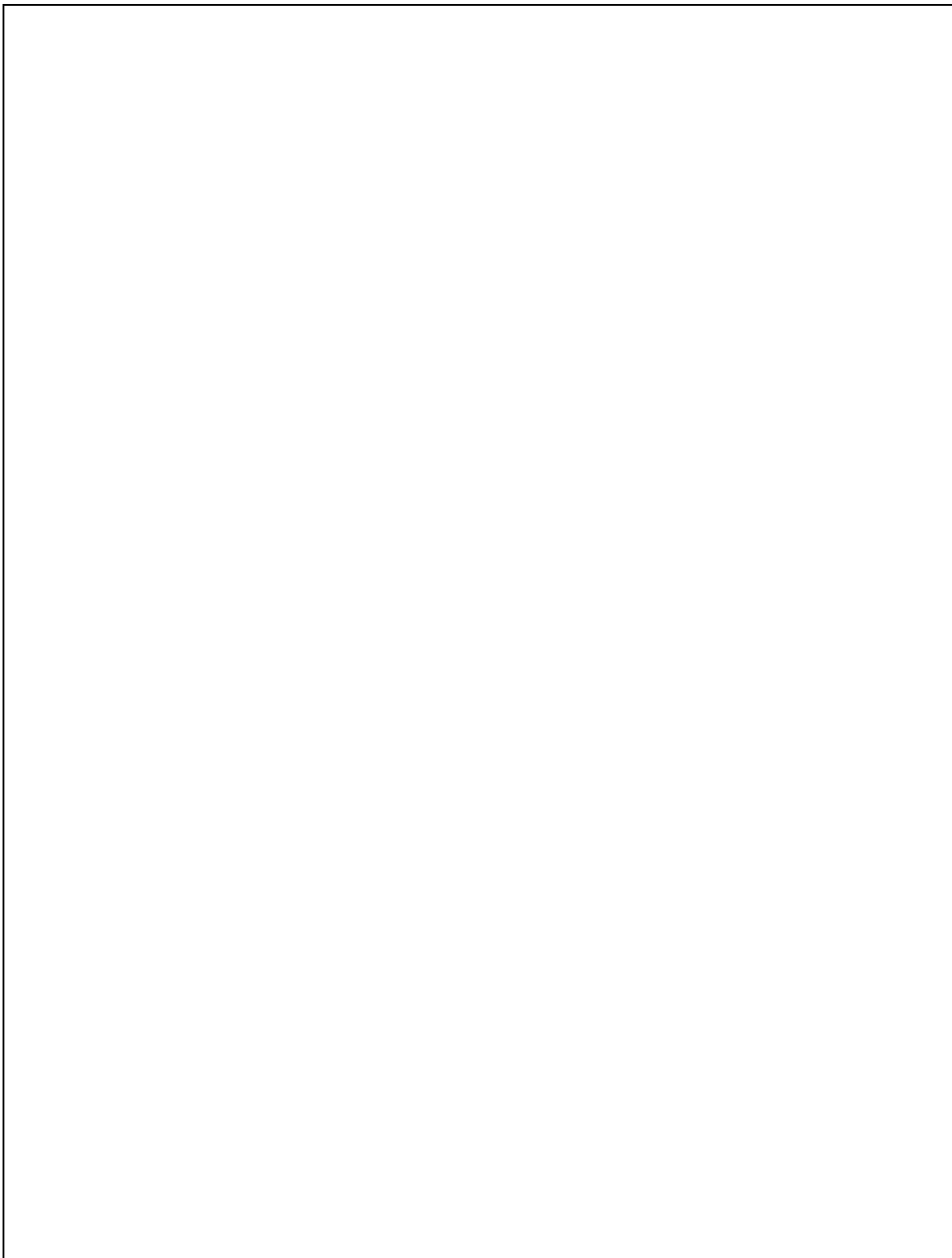
Roll No.: **A19**

Section: **Q5E06**

Department of Business Analytics

Lovely Professional University

Submitted To: Mr. Robin Tyagi



Objective

The main objective of this lab experiment was to understand how partitioning works in Apache Hive and to implement both Static and Dynamic partitioning techniques. This experiment was done on a Cloudera QuickStart virtual machine running Hadoop and Hive. The goal was to create Hive databases, load data from local text files, and then organise that data into partitions based on the 'state' column. This kind of partitioning is very useful in real-world big data applications because it helps in faster querying by scanning only the relevant partition rather than the entire dataset.

Introduction to Hive Partitioning

Apache Hive is a data warehousing solution built on top of Hadoop. It allows users to write SQL-like queries (called HiveQL) to process and analyse large datasets stored in HDFS. One of the most powerful features of Hive is partitioning, which allows the data to be split into smaller chunks based on the value of a particular column.

When a table is partitioned, Hive creates a separate directory for each unique value of the partition column. This means when a query filters data using that column, Hive only reads the relevant directory instead of scanning the full table. This significantly improves query performance, especially for very large datasets.

There are two types of partitioning in Hive:

1. Static Partitioning: The partition values are manually specified at the time of loading data. This is done when you already know the partition value beforehand.
2. Dynamic Partitioning: Hive automatically determines the partition values based on the data. The data is inserted from another table, and Hive creates partitions on the fly.

Tools and Environment Used

The following tools and environment were used for this experiment:

- Platform: Cloudera QuickStart VM 5.13.0 running on Oracle VirtualBox
- Big Data Framework: Apache Hadoop with Apache Hive
- File System: HDFS (Hadoop Distributed File System)
- Text Editor: gedit (for creating CSV data files on the desktop)
- Query Language: HiveQL (Hive Query Language)

Procedure

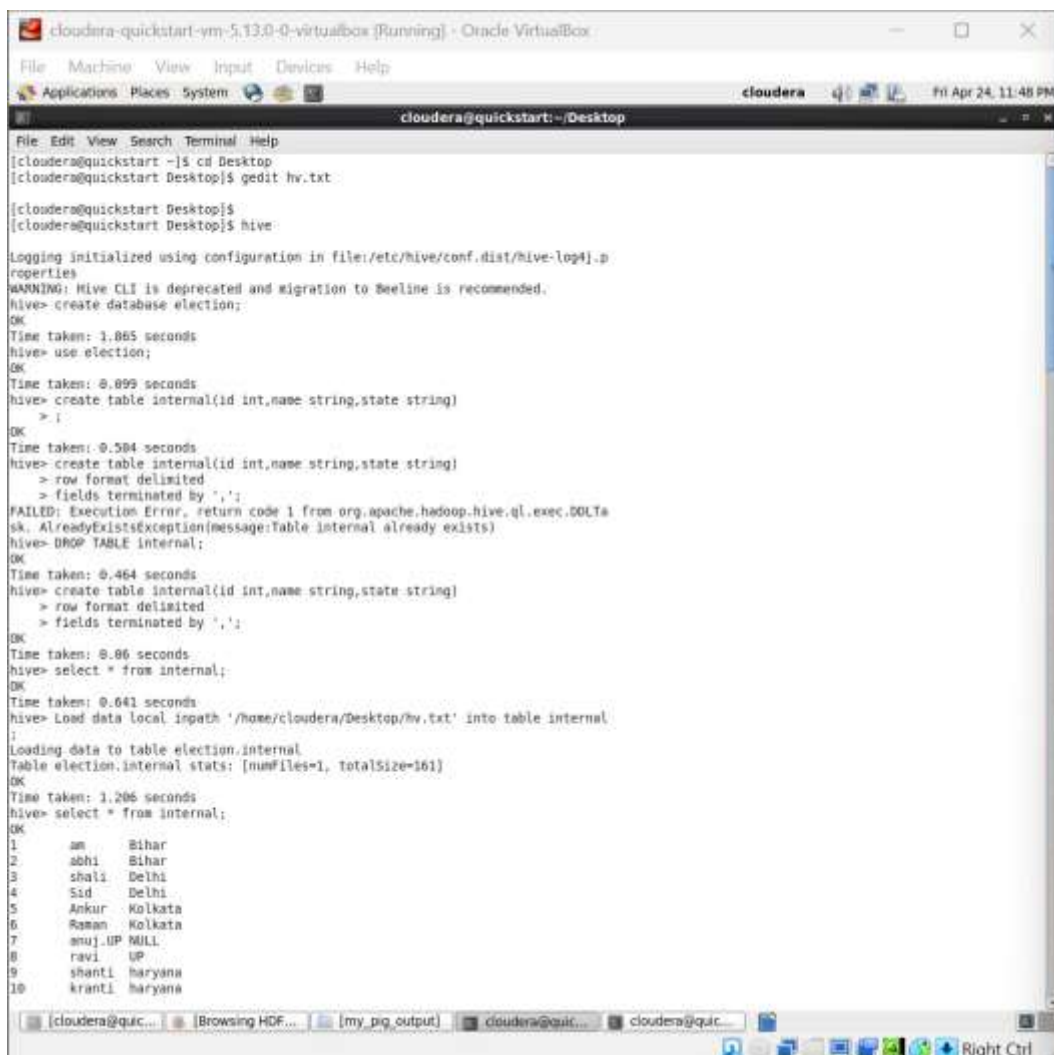
Part 1: Creating an Internal Table and Loading Data

The first step was to open the terminal on the Cloudera VM and create a dataset. A simple text file named hv.txt was created on the Desktop using the gedit editor. This file

contained 10 rows of data with three columns: id, name, and state, where all values were separated by commas.

After creating the data file, the Hive shell was opened by typing 'hive' in the terminal. A new database named 'election' was created using the CREATE DATABASE command. Then, an internal table named 'internal' was created with three columns: id (integer), name (string), and state (string). The table used comma as the field delimiter.

Initially, there was an error because the table named 'internal' already existed from a previous attempt. So the table was dropped using the DROP TABLE command and then recreated. After that, the data from hv.txt was loaded into the table using the LOAD DATA LOCAL INPATH command. A SELECT * query confirmed that all 10 records were successfully loaded into the table.



```
cloudera-quickstart-vm-5.13.0-0-virtualbox [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Applications Places System cloudera Fri Apr 24, 11:48 PM
cloudera@quickstart: ~/Desktop
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ cd Desktop
[cloudera@quickstart Desktop]$ gedit hv.txt
[cloudera@quickstart Desktop]$
[cloudera@quickstart Desktop]$ hive
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.p
roperties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> create database election;
OK
Time taken: 1.065 seconds
hive> use election;
OK
Time taken: 0.099 seconds
hive> create table internal(id int,name string,state string)
> ;
OK
Time taken: 0.504 seconds
hive> create table internal(id int,name string,state string)
> row format delimited
> fields terminated by ',';
FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.ql.exec.DDLEx
ec. AlreadyExistsException(message:Table internal already exists)
hive> DROP TABLE internal;
OK
Time taken: 0.464 seconds
hive> create table internal(id int,name string,state string)
> row format delimited
> fields terminated by ',';
OK
Time taken: 0.06 seconds
hive> select * from internal;
OK
Time taken: 0.641 seconds
hive> Load data local inpath '/home/cloudera/Desktop/hv.txt' into table internal
;
Loading data to table election.internal
Table election.internal stats: [numFiles=1, totalSize=161]
OK
Time taken: 1.206 seconds
hive> select * from internal;
OK
1      am      Bihar
2      abhi    Bihar
3      shali    Delhi
4      Sid     Delhi
5      Ankur   Kolkata
6      Ramon   Kolkata
7      anuj    UP
8      ravi    UP
9      shanti  haryana
10     kranti   haryana
```

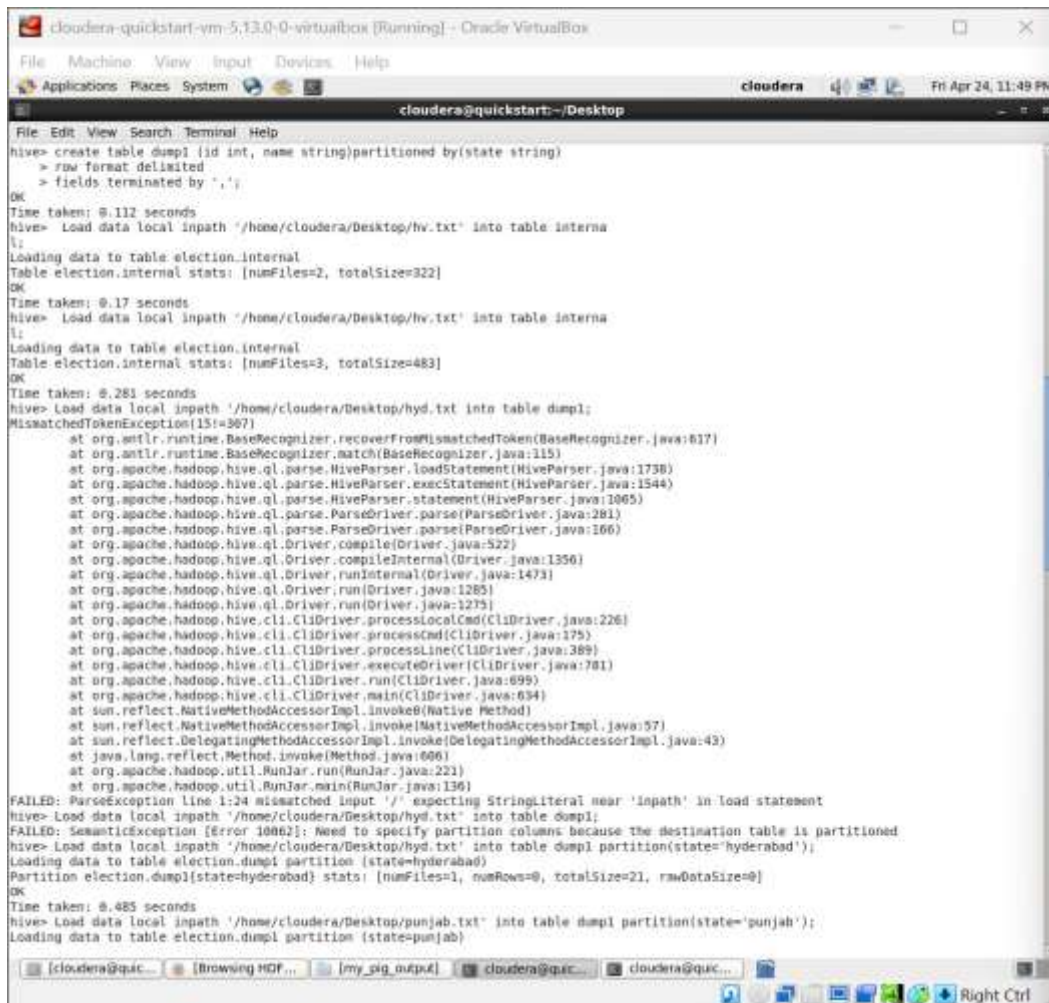
Screenshot 1: Creating the database, internal table, and loading data from hv.txt into Hive

Part 2: Static Partitioning

For static partitioning, a new partitioned table named 'dump1' was created. This table had two regular columns (id and name) and one partition column (state). Separate data files were created for each state: hyd.txt for Hyderabad, punjab.txt for Punjab, and goa.txt for Goa. Each file contained only the id and name columns, since state was the partition key.

While loading data, there was first an error because the LOAD DATA command was written incorrectly without quotes around the file path. After correcting the syntax, another error appeared — Hive refused to load data without specifying the partition value, because the destination table was a partitioned table. This is the expected behaviour in static partitioning.

The correct command was: `LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hyd.txt' INTO TABLE dump1 PARTITION(state='hyderabad')`. This same approach was used for punjab and goa as well. Each load operation created a separate partition directory in HDFS for that state.



```
cloudera-quickstart-vm-5.13.0-0-virtualbox [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Applications Places System cloudera Fri Apr 24, 11:49 PM
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
hive> create table dump1 (id int, name string) partitioned by (state string)
> row format delimited
> fields terminated by ','
OK
Time taken: 0.112 seconds
hive> load data local inpath '/home/cloudera/Desktop/hv.txt' into table interna
l;
Loading data to table election.internal
Table election.internal stats: [numFiles=2, totalSize=322]
OK
Time taken: 0.17 seconds
hive> load data local inpath '/home/cloudera/Desktop/hv.txt' into table interna
l;
Loading data to table election.internal
Table election.internal stats: [numFiles=3, totalSize=483]
OK
Time taken: 0.281 seconds
hive> load data local inpath '/home/cloudera/Desktop/hyd.txt into table dump1;
MismatchedTokenException(151:367)
    at org.apache.hadoop.hive.ql.parse.BaseParser.parse(BaseParser.java:115)
    at org.apache.hadoop.hive.ql.parse.HiveParser.parse(HiveParser.java:1738)
    at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1544)
    at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1544)
    at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1544)
    at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1544)
    at org.apache.hadoop.hive.ql.Driver.compile(Driver.java:522)
    at org.apache.hadoop.hive.ql.Driver.compileInternal(Driver.java:1350)
    at org.apache.hadoop.hive.ql.Driver.runInternal(Driver.java:1473)
    at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1285)
    at org.apache.hadoop.hive.ql.Driver.run(Driver.java:1275)
    at org.apache.hadoop.hive.cli.CliDriver.processLocalCmd(CliDriver.java:226)
    at org.apache.hadoop.hive.cli.CliDriver.processCmd(CliDriver.java:175)
    at org.apache.hadoop.hive.cli.CliDriver.processLine(CliDriver.java:389)
    at org.apache.hadoop.hive.cli.CliDriver.executeDriver(CliDriver.java:701)
    at org.apache.hadoop.hive.cli.CliDriver.run(CliDriver.java:699)
    at org.apache.hadoop.hive.cli.CliDriver.main(CliDriver.java:634)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:57)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:606)
    at org.apache.hadoop.util.RunJar.run(RunJar.java:221)
    at org.apache.hadoop.util.RunJar.main(RunJar.java:136)
FAILED: ParseException line 1:24 mismatched input '/' expecting StringLiteral near 'inpath' in load statement
hive> load data local inpath '/home/cloudera/Desktop/hyd.txt' into table dump1;
FAILED: SemanticException [error 10002]: Need to specify partition columns because the destination table is partitioned
hive> load data local inpath '/home/cloudera/Desktop/hyd.txt' into table dump1 partition(state='hyderabad');
Loading data to table election.dump1 partition (state=hyderabad)
Partition election.dump1[state=hyderabad] stats: [numFiles=1, numRows=0, totalSize=21, rawDataSize=0]
OK
Time taken: 0.485 seconds
hive> load data local inpath '/home/cloudera/Desktop/punjab.txt' into table dump1 partition(state='punjab');
Loading data to table election.dump1 partition (state=punjab)
```

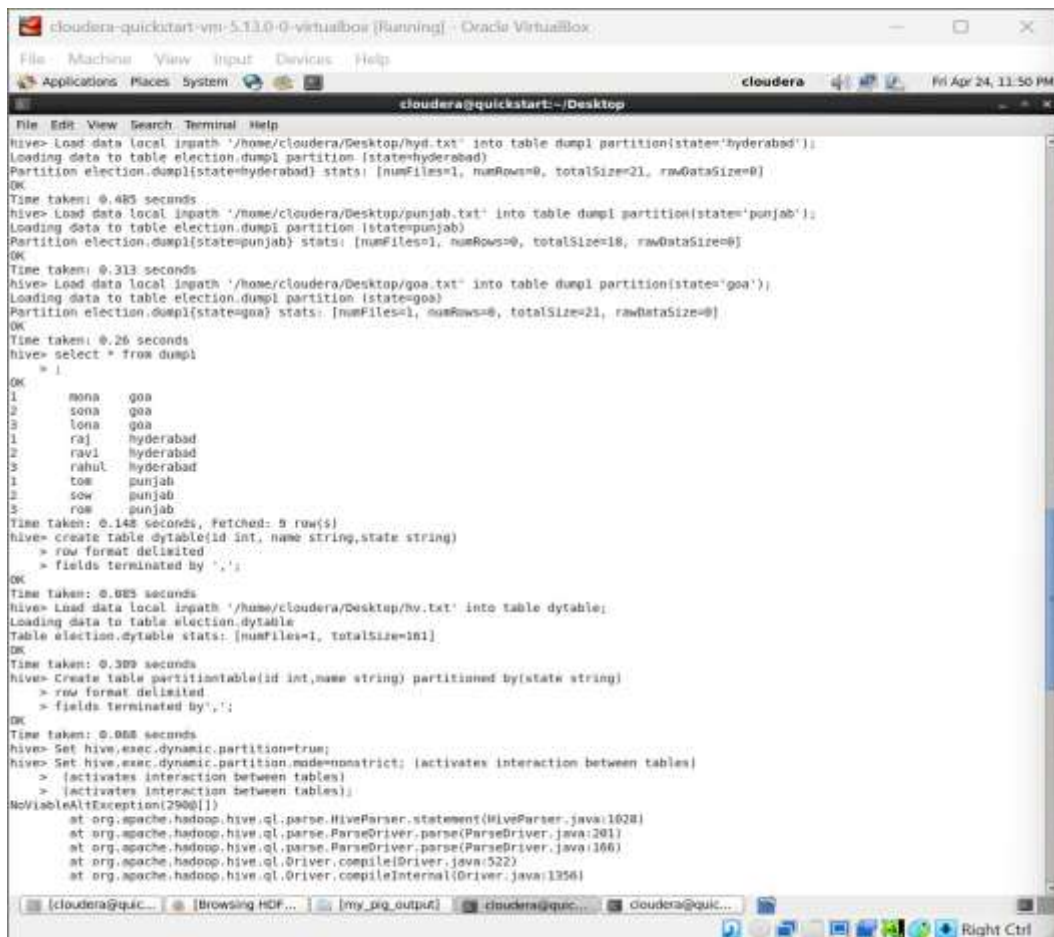
Screenshot 2: Static partitioning — loading state-wise data files into dump1 with explicit partition values

Part 3: Querying Static Partitioned Table and Setting Up Dynamic Partitioning

After loading all three state-wise files, a `SELECT *` query was run on the `dump1` table. The output showed 9 rows — 3 each for Goa, Hyderabad, and Punjab — confirming that all partitions were loaded correctly. The data was physically stored in separate HDFS directories based on the state value.

Next, to demonstrate dynamic partitioning, a staging table named 'dytable' was created with three columns: `id`, `name`, and `state`. The same `hv.txt` file (which had data for Bihar, Delhi, Kolkata, UP, and Haryana states) was loaded into this staging table. This table would serve as the source for the dynamic partitioning insert.

A new partitioned table called 'partitiontable' was then created. To enable dynamic partitioning, two Hive settings were configured: `hive.exec.dynamic.partition` was set to `true`, and `hive.exec.dynamic.partition.mode` was set to `nonstrict`. The `nonstrict` mode is required when all partition columns are dynamic (i.e., no static partition value is specified in the query). There was initially a syntax error in the `set` command because of extra text in the line, but it was corrected.



```
cloudera-quickstart-vm-5.13.0-0-virtualbox [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Applications Places System
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
hive> load data local inpath '/home/cloudera/Desktop/hyd.txt' into table dump1 partition(state='hyderabad');
Loading data to table election.dump1 partition (state=hyderabad)
Partition election.dump1(state=hyderabad) stats: [numFiles=1, numRows=0, totalSize=21, rawDataSize=0]
OK
Time taken: 0.483 seconds
hive> load data local inpath '/home/cloudera/Desktop/punjab.txt' into table dump1 partition(state='punjab');
Loading data to table election.dump1 partition (state=punjab)
Partition election.dump1(state=punjab) stats: [numFiles=1, numRows=0, totalSize=18, rawDataSize=0]
OK
Time taken: 0.313 seconds
hive> load data local inpath '/home/cloudera/Desktop/goa.txt' into table dump1 partition(state='goa');
Loading data to table election.dump1 partition (state=goa)
Partition election.dump1(state=goa) stats: [numFiles=1, numRows=0, totalSize=21, rawDataSize=0]
OK
Time taken: 0.26 seconds
hive> select * from dump1
*
1      mona      goa
2      sona      goa
3      lona      goa
1      raj       hyderabad
2      ravi      hyderabad
3      rahul     hyderabad
1      tom       punjab
2      sow       punjab
3      row       punjab
Time taken: 0.148 seconds, fetched: 9 row(s)
hive> create table dytable(id int, name string, state string)
> row format delimited
> fields terminated by ',';
OK
Time taken: 0.085 seconds
hive> load data local inpath '/home/cloudera/Desktop/hv.txt' into table dytable;
Loading data to table election.dytable
Table election.dytable stats: [numFiles=1, totalSize=161]
OK
Time taken: 0.309 seconds
hive> create table partitiontable(id int, name string) partitioned by(state string)
> row format delimited
> fields terminated by ',';
OK
Time taken: 0.068 seconds
hive> set hive.exec.dynamic.partition=true;
hive> set hive.exec.dynamic.partition.mode=nonstrict; (activates interaction between tables)
> (activates interaction between tables)
> (activates interaction between tables)
java.lang.RuntimeException: java.lang.IllegalArgumentException: Invalid argument for method org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1628)
at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1628)
at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1628)
at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1628)
at org.apache.hadoop.hive.ql.parse.HiveParser.parseStatement(HiveParser.java:1628)
at org.apache.hadoop.hive.ql.Driver.compileInternal(Driver.java:1356)
```

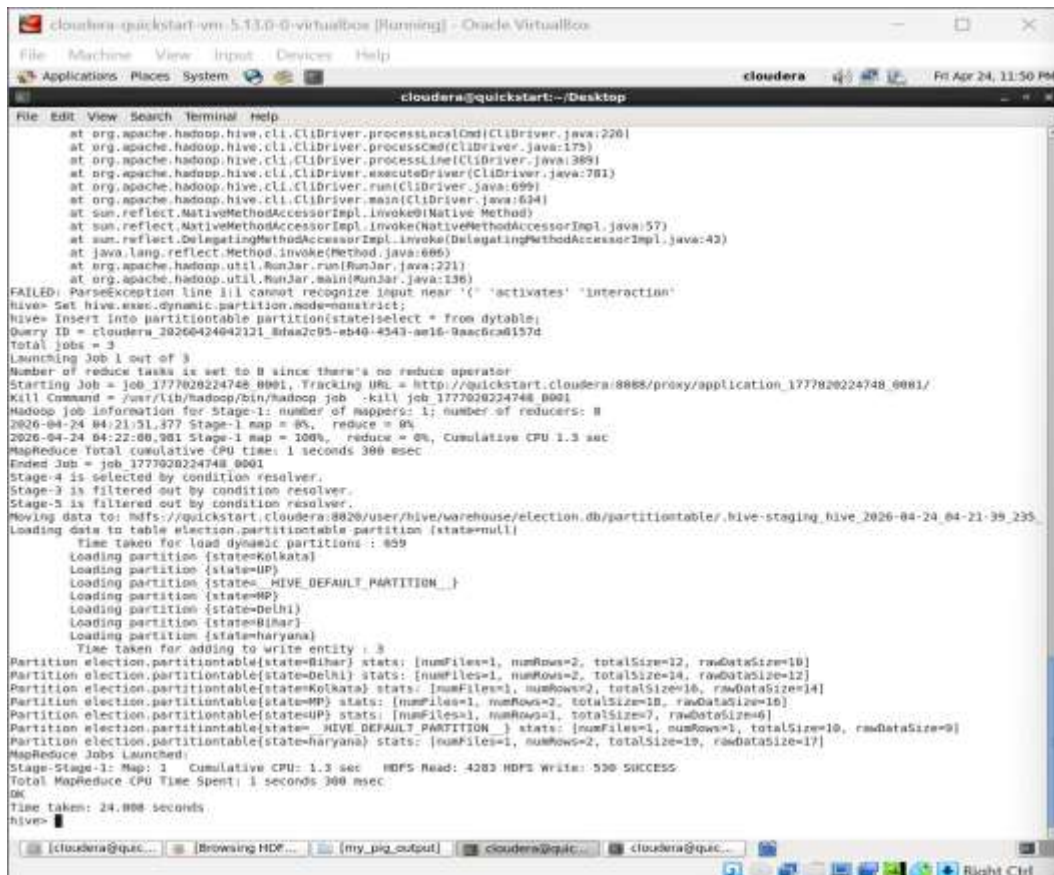
Screenshot 3: `SELECT` query results for static partitioned table, and setup of dynamic partitioning settings

Part 4: Dynamic Partitioning — Execution and Output

Once the dynamic partitioning settings were in place, the following INSERT command was executed: INSERT INTO partitiontable PARTITION(state) SELECT * FROM dytable. This single command triggered a MapReduce job that read all data from the dytable and automatically created partitions based on the distinct values found in the state column.

The MapReduce job completed successfully. The output showed that the following partitions were dynamically created: Bihar, Delhi, Kolkata, MP (Madhya Pradesh), UP, Haryana, and one default partition (_HIVE_DEFAULT_PARTITION_) which is created when a row has a NULL value in the state column. Looking back at the original data in hv.txt, there was indeed one record (anuj) that had UP as state but was split in the file such that it appeared as NULL.

Each partition showed the number of rows and data size: Bihar had 2 rows, Delhi had 2 rows, Kolkata had 2 rows, Haryana had 2 rows, UP had 1 row, and MP had 1 row. The total MapReduce operation took approximately 24 seconds. The entire process was fully automated — no manual specification of partition values was required, which is the main advantage of dynamic partitioning.



```
cloudera-quickstart-vm: 5.13.0-0-virtualbox [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Applications Places System cloudera Fri Apr 24, 11:50 PM

cloudera@quickstart:~/Desktop
File Edit View Search Terminal help
at org.apache.hadoop.hive.cli.CliDriver.processLocalCmd(CliDriver.java:220)
at org.apache.hadoop.hive.cli.CliDriver.processCmd(CliDriver.java:175)
at org.apache.hadoop.hive.cli.CliDriver.processLine(CliDriver.java:389)
at org.apache.hadoop.hive.cli.CliDriver.executeDriver(CliDriver.java:701)
at org.apache.hadoop.hive.cli.CliDriver.run(CliDriver.java:699)
at org.apache.hadoop.hive.cli.CliDriver.main(CliDriver.java:634)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethod)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:57)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
at java.lang.reflect.Method.invoke(Method.java:606)
at org.apache.hadoop.util.RunJar.run(RunJar.java:221)
at org.apache.hadoop.util.RunJar.main(RunJar.java:136)
FAILED: ParseException line 1:1 cannot recognize input near '(' 'activates' 'interaction'
hive> Set hive.exec.dynamic.partition.mode=nonstrict;
hive> Insert into partitiontable partition(state)select * from dytable;
Query ID = cloudera_20260424042121_8fba2c95-bb40-4543-aa16-9aac6ca6157d
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1777020224748_0001, Tracking URL = http://quickstart.cloudera:8888/proxy/application_1777020224748_0001/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777020224748_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-04-24 04:21:31,377 Stage-1 map = 0%, reduce = 0%
2026-04-24 04:22:00,901 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.3 sec
MapReduce total cumulative CPU time: 1 seconds 300 msec
Ended Job = job_1777020224748_0001
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8820/user/hive/warehouse/election.db/partitiontable/hive-staging_hive_2026-04-24_04-21-39_235
Loading data to table election.partitiontable partition (state=null)
Time taken for load dynamic partitions : 059
Loading partition {state=Kolkata}
Loading partition {state=UP}
Loading partition {state=_HIVE_DEFAULT_PARTITION_}
Loading partition {state=MP}
Loading partition {state=Delhi}
Loading partition {state=Bihar}
Loading partition {state=Haryana}
Time taken for adding to write entity : 3
Partition election.partitiontable(state=Bihar) stats: [numFiles=1, numRows=2, totalSize=12, rawDataSize=10]
Partition election.partitiontable(state=Delhi) stats: [numFiles=1, numRows=2, totalSize=14, rawDataSize=12]
Partition election.partitiontable(state=Kolkata) stats: [numFiles=1, numRows=2, totalSize=16, rawDataSize=14]
Partition election.partitiontable(state=MP) stats: [numFiles=1, numRows=2, totalSize=18, rawDataSize=16]
Partition election.partitiontable(state=UP) stats: [numFiles=1, numRows=1, totalSize=7, rawDataSize=6]
Partition election.partitiontable(state=_HIVE_DEFAULT_PARTITION_) stats: [numFiles=1, numRows=1, totalSize=10, rawDataSize=9]
Partition election.partitiontable(state=Haryana) stats: [numFiles=1, numRows=2, totalSize=19, rawDataSize=17]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 1.3 sec HDFS Read: 4203 HDFS Write: 530 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 300 msec
OK
Time taken: 24.808 seconds
hive>
```

Screenshot 4: Dynamic partitioning execution — MapReduce job loading state-wise partitions automatically

Errors Encountered and How They Were Resolved

During this experiment, a few common errors were encountered. Understanding these errors is an important part of learning:

1. `AlreadyExistsException`: When trying to create the 'internal' table again, Hive threw an error saying the table already exists. This was resolved by first dropping the existing table with `DROP TABLE internal`, and then recreating it.
2. `MismatchedTokenException`: This occurred when the file path in the `LOAD DATA` command was typed without quotes. Hive's parser could not interpret the forward slashes. The fix was to wrap the file path in single quotes.
3. `SemanticException` (Error 10062): When trying to load data into `dump1` without specifying a partition value, Hive returned this error. It is a mandatory requirement for static partitioning to explicitly mention the `PARTITION` clause with the value.
4. `NoViableAltException`: During dynamic partition setup, extra text was accidentally appended to the `SET` command. The issue was corrected by running the two `SET` commands separately and cleanly.

Results and Observations

The experiment was completed successfully. Both static and dynamic partitioning were implemented in Apache Hive on the Cloudera VM. In static partitioning, data for three states (Hyderabad, Punjab, and Goa) was loaded manually into separate partitions using explicit partition values. In dynamic partitioning, the system automatically created 7 partitions from the source table using a single `INSERT` statement driven by a MapReduce job.

One key observation was that in dynamic partitioning, if a row has a `NULL` value in the partition column, Hive places it in a special default partition called `__HIVE_DEFAULT_PARTITION__`. This is an important behaviour to be aware of when dealing with real-world dirty data.

Also, dynamic partitioning is more flexible and powerful for large datasets where partition values are not known in advance, while static partitioning gives more control and is simpler for smaller, well-defined datasets.

Conclusion

This experiment gave a clear hands-on understanding of how partitioning works in Apache Hive. Partitioning is one of the most important optimisation techniques in any big data processing system. By dividing data into smaller manageable directories based on column values, query performance improves drastically because the system does not have to scan the entire dataset.

Static partitioning is straightforward but requires manual effort for each load. Dynamic partitioning, on the other hand, automates this process using MapReduce and is ideal when dealing with large volumes of data. Overall, this lab exercise reinforced the

importance of proper data organisation in big data ecosystems and helped in understanding the internal directory structure that Hive creates in HDFS for partitioned tables.